

Modelo Cosmológico Estándar

Pau Gallego Cisteró

Abril 2026

Tarea 2

4. Se pide generar un conjunto de $N = 20$ partículas en 1D con densidad promedio $\bar{\rho}$ y añadir una perturbación sinusoidal pequeña $\epsilon \sin 2\pi x_0$. Se pide la evolución temporal y mostrar gráficamente cómo se acumulan partículas en algunas zonas. Ayuda: se puede dividir el segmento en 10 compartimentos y ver cómo se van acumulando en algunos y otros se vacían.

En primera instancia se muestra el código al que se ha llegado a mano basado en la ecuación de Jean y el resultado del histograma conseguido. Posteriormente se muestra un código que se ha obtenido con IA basado en el inicial, con el objetivo de obtener un gráfico de trayectorias y poder visualizar la evolución temporal en contraste con el histograma, como se comentará, este segundo código muestra una discrepancia a pesar de tener el mismo esqueleto que el original.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 20
5 G = 6.674e-11
6 L_min = 0
7 L_max = 1
8 t0 = 0.1
9 t_f = 10
10 ts = np.linspace(t0, t_f, N)
11 rho_0 = 1.5e10 # cte en rho = cte * a^-3, grande para compensar
12               G
13 x0 = np.linspace(L_min, L_max, N, endpoint=False)
14 epsilon = 0.1 # cuanto se desplazan las particulas
15 delta = epsilon * np.sin(2 * np.pi * x0) # Perturbacion inicial
16               pequenya
17 delta_dot = 0.0
18
19 def ecu_densidad(delta, delta_dot, t):
20     a = (t)**(2/3) # Factor de escala
21     H = (2/3) * (1/t) # Hubble
```

```

20     rho = rho_0 / (a**3) # Densidad media
21
22     d2_delta = 4 * np.pi * G * rho * delta - 2 * H * delta_dot
23     return d2_delta
24
25 dt = 0.01
26
27 fig, axs = plt.subplots(1, 3, figsize=(15, 5), sharey=True)
28
29 # Creamos una figura con 1 fila y 3 columnas
30 fig, axs = plt.subplots(1, 3, figsize=(15, 5), sharey=True)
31
32
33 momentos = [t0, 5, 10]
34
35 for i, t in enumerate(momentos):
36
37     while t0 < t:
38         ddelta = ecu_densidad(delta, delta_dot, t0)
39         delta_dot += ddelta * dt
40         delta += delta_dot * dt
41         t0 += dt
42
43     posiciones = (x0 + delta) % L_max
44
45     axs[i].hist(posiciones, bins=10, range=(0,1),
46                color='royalblue', edgecolor='black', rwidth
47                =0.85)
48
49     # Titulos individuales en cada subplot
50     axs[i].set_title(f'Tiempo t = {t}', fontsize=14)
51     axs[i].set_xlabel('Posicion', fontsize=12)
52     axs[i].grid(axis='y', alpha=0.3)
53
54     axs[0].set_ylabel('Numero de particulas', fontsize=12)
55
56 plt.tight_layout() # Ajusta automaticamente los margenes para
                    # que no se pisen las letras
plt.show()

```

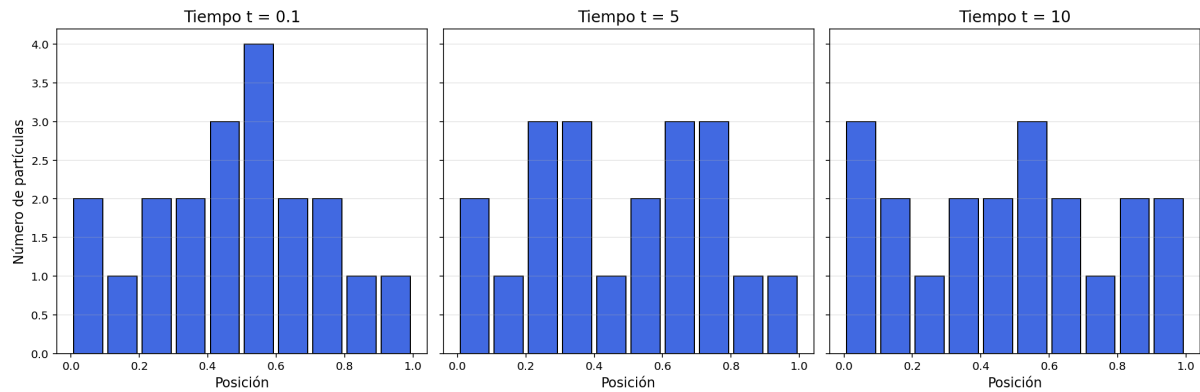


Figure 1: Histogramas para distintos tiempos - primer modelo

Este es el código hecho con IA;

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # --- PARAMETROS INICIALES ---
5 N = 20
6 G = 6.674e-11
7 L_min = 0
8 L_max = 1
9 t0 = 0.1
10 t_f = 10
11 ts = np.linspace(t0, t_f, N)
12 rho_0 = 1.5e10 # Densidad aumentada para compensar G
13 x0 = np.linspace(L_min, L_max, N, endpoint=False)
14 epsilon = 0.1
15
16 # Perturbacion inicial
17 delta = epsilon * np.sin(2 * np.pi * x0)
18 delta_dot = np.zeros(N) # debe ser un array de ceros, no un
    numero
19
20 # --- ECUACION DE JEANS ---
21 def ecu_densidad(delta, delta_dot, t):
22     a = (t)**(2/3)
23     H = (2/3) * (1/t)
24     rho = rho_0 / (a**3)
25
26     d2_delta = 4 * np.pi * G * rho * delta - 2 * H * delta_dot
27     return d2_delta
28
29 # --- INTEGRACION Y GUARDADO DE HISTORIAL ---
30 dt = 0.01
31 t_actual = t0
32
33 momentos = [t0, 5, 10]
34 datos_histogramas = [] # Guardara las posiciones para los 3
    momentos exactos

```

```

35
36 historial_t = [] # Guardara todos los instantes de tiempo
37 historial_x = [] # Guardara todas las posiciones para trazar las
    lineas
38
39 while t_actual <= t_f + dt:
40
41     # Calculamos la posicion en este instante
42     # Usamos x0 + delta directamente para el grafico lineal para
    evitar
43     # que las lineas den saltos visuales feos al cruzar el borde
    L_max
44     pos_actuales = x0 + delta
45
46     # 2. Guardamos datos para el grafico de lineas (el "video")
47     historial_t.append(t_actual)
48     historial_x.append(pos_actuales.copy())
49
50     # Guardamos datos para los histogramas (las 3 "fotos")
51     # Si el tiempo actual esta muy cerca de uno de nuestros "
    momentos", lo guardamos
52     for m in momentos:
53         if abs(t_actual - m) < dt / 2:
54             datos_histogramas.append((m, pos_actuales % L_max))
55
56     # Avanzamos la fisica al siguiente instante
57     ddelta = ecu_densidad(delta, delta_dot, t_actual)
58     delta_dot += ddelta * dt
59     delta += delta_dot * dt
60     t_actual += dt
61
62 # Convertimos las listas a matrices de numpy para que sea facil
    graficarlas
63 historial_t = np.array(historial_t)
64 historial_x = np.array(historial_x)
65
66 # --- VISUALIZACION ---
67 fig = plt.figure(figsize=(12, 10))
68
69 # GRAFICO 1: Trayectorias (Grafico lineal como el de tu dibujo)
70 # Ocupa toda la fila superior (posicion 1 de una cuadrricula
    imaginaria 2x1)
71 ax_trayectoria = plt.subplot(2, 1, 1)
72
73 # Dibujamos una linea por cada una de las N particulas
74 for i in range(N):
75     ax_trayectoria.plot(historial_t, historial_x[:, i],
        linewidth=1.5)
76
77 ax_trayectoria.set_title('Evolucion temporal de las trayectorias
    (Acumulacion)', fontsize=15)

```

```

78 ax_trayectoria.set_xlabel('Tiempo (t)', fontsize=12)
79 ax_trayectoria.set_ylabel('Posicion (x)', fontsize=12)
80 ax_trayectoria.grid(True, alpha=0.3)
81 ax_trayectoria.set_xlim(t0, t_f)
82
83 # GRAFICOS 2, 3 y 4: Histogramas (Fila inferior)
84 # Se posicionan en las celdas 4, 5 y 6 de una cuadrícula 2x3
85 for i, (tiempo_foto, pos_foto) in enumerate(datos_histogramas):
86     ax_hist = plt.subplot(2, 3, i + 4)
87     ax_hist.hist(pos_foto, bins=10, range=(0,1), color='
royalblue', edgecolor='black', rwidth=0.85)
88
89     ax_hist.set_title(f'Tiempo t = {tiempo_foto:.1f}', fontsize
=12)
90     ax_hist.set_xlabel('Posicion', fontsize=11)
91     if i == 0:
92         ax_hist.set_ylabel('Numero de particulas', fontsize=11)
93     ax_hist.grid(axis='y', alpha=0.3)
94
95 plt.tight_layout() # Ajusta espacios automaticamente
96 plt.show()

```

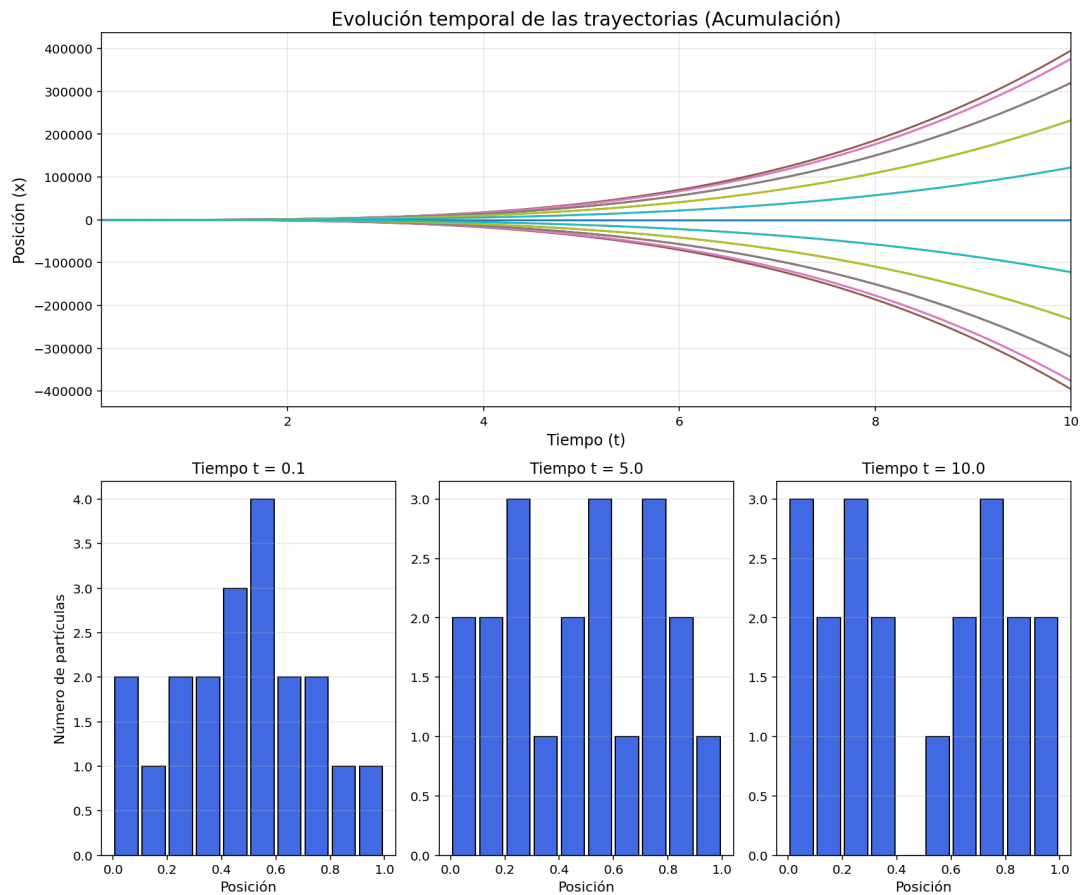


Figure 2: Gráfico de trayectorias + histograma erroneo - segundo modelo

La discrepancia visual entre los histogramas se debe a la sensibilidad del sistema frente a errores numéricos de punto flotante en el bucle $t_0 < t$. Esta imprecisión matemática obliga al primer código a calcular pasos temporales extra. Dado que el modelo presenta un colapso gravitacional exponencial con velocidades extremas, una sola iteración adicional mueve las partículas a enormes distancias. Al aplicar la condición periódica, estos desplazamientos masivos alteran drásticamente la posición final proyectada, reubicando caóticamente las partículas en barras completamente distintas del histograma, al contrario que la Figura 1, que es la representación correcta.

Uso de IA

Se ha usado IA para encontrar la manera correcta de escribir rho, para poner títulos individuales a cada histograma, pulir la eficiencia del while y para la generación del segundo modelo.

References

- [1] Sanz Moreno M. *Modelo Cosmológico Estandar - Tarea 2, 1r draft estático erroneo*. Abril 2026.
- [2] DataCamp. *Histogramas con Matplotlib*. <https://www.datacamp.com/es/tutorial/histograms-matplotlib>